

AIAC ASSIGNMENT – 8

NAME : E.ANJANI

HNO:2303A52419

BATCH:32

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

```
Task1.py Task2.py Task3.py Task4.py Task5.py Task6.py Task7.py Task8.py Task9.py
Task1.py > is_valid_username
1 def is_valid_username(username):
2     if len(username) < 5 or len(username) > 15:
3         return False
4     if not username.isalnum():
5         return False
6     if username[0].isdigit():
7         return False
8     if ' ' in username:
9         return False
10    return True
11    assert is_valid_username("User123") == True
12    assert is_valid_username("123User") == False
13    assert is_valid_username("Us er") == False
14    print("All test cases passed!")

PS D:\courses\AIAC\Lab_Ass_8> & C:/Python314/python.exe d:/courses/AIAC/Lab_Ass_8/Task1.py
All test cases passed!
PS D:\courses\AIAC\Lab_Ass_8>
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

```
Task2.py > classify_value
1 def classify_value(x):
2     if not isinstance(x, (int, float)):
3         return "Not a number"
4     for i in range(1):
5         if x == 0:
6             return "Zero"
7         if x % 2 == 0:
8             return "Even"
9         else:
10            return "Odd"
11    assert classify_value(8) == "Even"
12    assert classify_value(7) == "Odd"
13    assert classify_value(0) == "Zero"
14    assert classify_value("Hello") == "Not a number"
15    print("All test cases passed!")

PS D:\courses\AIAC\Lab_Ass_8> & C:/Python314/python.exe d:/courses/AIAC/Lab_Ass_8/Task2.py
All test cases passed!
PS D:\courses\AIAC\Lab_Ass_8>
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.

```
Task3.py > ...
1 def is_palindrome(text):
2     #remove cases , spaces and punctuation
3     cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
4     return cleaned_text == cleaned_text[::-1]
5 assert is_palindrome("Madam") == True
6 assert is_palindrome("A man a plan a canal Panama") == True
7 assert is_palindrome("Python") == False
8 print("All test cases passed!")

PS D:\courses\AIAC\Lab_Ass_8> & C:/Python314/python.exe d:/courses/AIAC/Lab_Ass_8/Task3.py
All test cases passed!
PS D:\courses\AIAC\Lab_Ass_8>
```

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

```
Task4.py > ...
1 def validate_email(email):
2     return False
3     # must not start or end with special characters
4     if not local_part[0].isalnum() or not local_part[-1].isalnum():
5         return False
6     if not domain_part[0].isalnum() or not domain_part[-1].isalnum():
7         return False
8     return True
9 assert validate_email("user@example.com") == True
10 assert validate_email("userexample.com") == False
11 assert validate_email("@gmail.com") == False
12 print("All test cases passed!")

PS D:\courses\AIAC\Lab_Ass_8> & C:/Python314/python.exe d:/courses/AIAC/Lab_Ass_8/Task4.py
All test cases passed!
PS D:\courses\AIAC\Lab_Ass_8>
```

Task 5 (Perfect Number Checker – Test Case Design)

- Function: Check if a number is a perfect number (sum of divisors = number).

```
Task5.py > ...
1 def perfectnumber(n):
2     if n <= 1:
3         return False
4     divisors = []
5     for i in range(1, n):
6         if n % i == 0:
7             divisors.append(i)
8     return sum(divisors) == n
9 assert perfectnumber(6) == True
10 assert perfectnumber(10) == False
11 assert perfectnumber(1) == False
12 assert perfectnumber(28) == True
13 print("All tests passed!")
14
```

```

PS D:\courses\AIAC\Lab_Ass_8> & C:/Python314/python.exe d:/courses/AIAC/Lab_Ass_8/Task5.py
All tests passed!
PS D:\courses\AIAC\Lab_Ass_8>

```

Task 6 (Abundant Number Checker – Test Case Design)

- Function: Check if a number is abundant (sum of divisors > number).
- Test Cases to Design:
 - o Normal case: 12 → True, 15 → False.
 - o Edge case: 1.
 - o Negative number case.
 - o Large case: 945.

```

Task6.py > TestAbundant > test_non_abundant
1  import unittest
2  def abundant(n):
3      if n <= 1:
4          return False
5      divisors = []
6      for i in range(1, n):
7          if n % i == 0:
8              divisors.append(i)
9      return sum(divisors) > n
10 class TestAbundant(unittest.TestCase):
11     def test_abundant(self):
12         self.assertTrue(abundant(12))
13         self.assertTrue(abundant(18))
14         self.assertTrue(abundant(945))
15     def test_non_abundant(self):
16         self.assertFalse(abundant(1))
17         self.assertFalse(abundant(15))
18         self.assertFalse(abundant(-3))
19 if __name__ == '__main__':
20     unittest.main()
21
PS D:\courses\AIAC\Lab_Ass_8> & C:/Python314/python.exe d:/courses/AIAC/Lab_Ass_8/Task6.py
..
-----
Ran 2 tests in 0.000s

OK
PS D:\courses\AIAC\Lab_Ass_8>

```

Task 7 (Deficient Number Checker – Test Case Design)

- Function: Check if a number is deficient (sum of divisors < number).
- Test Cases to Design:
 - o Normal case: 8 → True, 12 → False.

- o Edge case: 1.
- o Negative number case.
- o Large case: 546.

Requirement: Validate correctness with pytest.

```

Task7.py > test_negativenumber

1  def deficient(n):
2      if n <= 1:
3          return False
4      divisors = []
5      for i in range(1, n):
6          if n % i == 0:
7              divisors.append(i)
8      return sum(divisors) < n
9  def test_deficient():
10     assert deficient(1) == False
11     assert deficient(12) == False
12     assert deficient(8) == True
13     assert deficient(546) == False
14  def test_negativenumber():
15     assert deficient(-3) == False
16  def test_edgecases():
17     assert deficient(0) == False
18     assert deficient(1) == False
19

PS D:\courses\AIAC\Lab_Ass_8> python -m pytest Task7.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\courses\AIAC\Lab_Ass_8
collected 3 items

Task7.py ... [100%]

===== 3 passed in 0.01s =====
PS D:\courses\AIAC\Lab_Ass_8>

```

Task 8 :

Write a function LeapYearChecker and validate its implementation using 10 pytest test cases

```
Task8.py > test_Leapyearchecker9
1 def Leapyearchecker(year):
2     if year % 4 == 0:
3         if year % 100 == 0:
4             if year % 400 == 0:
5                 return True
6             else:
7                 return False
8         else:
9             return True
10    else:
11        return False
12    #generate 10 test cases for the function
13    def test_Leapyearchecker():
14        assert Leapyearchecker(2000) == True
15        assert Leapyearchecker(1900) == False
16    def test_Leapyearchecker2():
17        assert Leapyearchecker(2004) == True
18        assert Leapyearchecker(2001) == False
19    def test_Leapyearchecker3():
20        assert Leapyearchecker(2012) == True
21        assert Leapyearchecker(2019) == False
22    def test_Leapyearchecker4():
23        assert Leapyearchecker(2020) == True
24        assert Leapyearchecker(2021) == False
25    def test_Leapyearchecker5():
26        assert Leapyearchecker(2024) == True
27        assert Leapyearchecker(2025) == False
28    def test_Leapyearchecker6():
29        assert Leapyearchecker(2028) == True
30        assert Leapyearchecker(2029) == False
31    def test_Leapyearchecker7():
32        assert Leapyearchecker(2032) == True
33        assert Leapyearchecker(2033) == False
34    def test_Leapyearchecker8():
35        assert Leapyearchecker(2036) == True
36        assert Leapyearchecker(2037) == False
37    def test_Leapyearchecker9():
38        assert Leapyearchecker(2040) == True
39        assert Leapyearchecker(2041) == False
40    def test_Leapyearchecker10():
41        assert Leapyearchecker(2044) == True
42        assert Leapyearchecker(2045) == False
43

PS D:\courses\AIAC\Lab_Ass_8> python -m pytest Task8.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\courses\AIAC\Lab_Ass_8
collected 10 items

Task8.py ..... [100%]

===== 10 passed in 0.02s =====
PS D:\courses\AIAC\Lab_Ass_8>
```

Task 9 :

Write a function SumOfDigits and validate its implementation using 7 pytest test cases.

```
Task9.py > ...
1 def sumofdigits(n):
2     total = 0
3     while n > 0:
4         total += n % 10
5         n //= 10
6     return total
7 def test_sumofdigits():
8     assert sumofdigits(123) == 6
9     assert sumofdigits(456) == 15
10    assert sumofdigits(789) == 24
11 def test_sumofdigits2():
12    assert sumofdigits(1000) == 1
13    assert sumofdigits(9999) == 36
14 def test_sumofdigits3():
15    assert sumofdigits(0) == 0
16    assert sumofdigits(5) == 5
17 def test_sumofdigits4():
18    assert sumofdigits(12345) == 15
19    assert sumofdigits(675) == 18
20 def test_sumofdigits5():
21    assert sumofdigits(999) == 27
22    assert sumofdigits(123456) == 21
23 def test_sumofdigits6():
24    assert sumofdigits(1111) == 4
25    assert sumofdigits(2222) == 8
26 def test_sumofdigits7():
27    assert sumofdigits(3333) == 12
28    assert sumofdigits(4444) == 16
29

PS D:\courses\AIAC\Lab_Ass_8> python -m pytest Task9.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\courses\AIAC\Lab_Ass_8
collected 7 items

Task9.py ..... [100%]

===== 7 passed in 0.02s =====
PS D:\courses\AIAC\Lab_Ass_8>
```

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate its implementation using 25 pytest test cases.

```

Task10.py > sortnumbers
1 def sortnumbers(numbers):
2     n = len(numbers)
3     for i in range(n):
4         for j in range(0, n - i - 1):
5             if numbers[j] > numbers[j + 1]:
6                 numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
7     return numbers
8
9 def test_sortnumbers():
10     assert sortnumbers([5, 2, 8, 1, 9]) == [1, 2, 5, 8, 9]
11
12 def test_sortnumbers2():
13     assert sortnumbers([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
14
15 def test_sortnumbers3():
16     assert sortnumbers([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
17
18 def test_sortnumbers4():
19     assert sortnumbers([1]) == [1]
20
21 def test_sortnumbers5():
22     assert sortnumbers([]) == []
23
24 def test_sortnumbers6():
25     assert sortnumbers([3, 1]) == [1, 3]
26
27 def test_sortnumbers7():
28     assert sortnumbers([2, 2]) == [2, 2]
29
30 def test_sortnumbers8():
31     assert sortnumbers([7, -1]) == [-1, 7]
32
33 def test_sortnumbers9():
34     assert sortnumbers([-5, -2]) == [-5, -2]
35
36 def test_sortnumbers10():
37     assert sortnumbers([0]) == [0]
38
39 def test_sortnumbers11():
40     assert sortnumbers([0, -1]) == [-1, 0]
41
42 def test_sortnumbers12():
43     assert sortnumbers([999999999]) == [999999999]
44
45 def test_sortnumbers13():
46     assert sortnumbers([-500000000]) == [-500000000]
47
48 def test_sortnumbers14():
49     assert sortnumbers([888888888]) == [888888888]
50
51 def test_sortnumbers15():
52     assert sortnumbers([-777777777]) == [-777777777]
53
54 def test_sortnumbers16():
55     assert sortnumbers([444444444]) == [444444444]
56
57 def test_sortnumbers17():
58     assert sortnumbers([-666666666]) == [-666666666]
59
60 def test_sortnumbers18():
61     assert sortnumbers([333333333]) == [333333333]
62
63 def test_sortnumbers19():
64     assert sortnumbers([-222222222]) == [-222222222]
65
66 def test_sortnumbers20():
67     assert sortnumbers([555555555]) == [555555555]
68
69 def test_sortnumbers21():
70     assert sortnumbers([-111111111]) == [-111111111]
71
72 def test_sortnumbers22():
73     assert sortnumbers([777777777]) == [777777777]
74
75 def test_sortnumbers23():
76     assert sortnumbers([-333333333]) == [-333333333]
77
78 def test_sortnumbers24():
79     assert sortnumbers([666666666]) == [666666666]
80
81 def test_sortnumbers25():
82     assert sortnumbers([-444444444]) == [-444444444]

```

PS D:\courses\AIAC\Lab_Ass_8> python -m pytest Task10.py

test session starts

platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\courses\AIAC\Lab_Ass_8
collected 25 items

Task10.py

[100%]

===== 25 passed in 0.04s =====

PS D:\courses\AIAC\Lab_Ass_8>

Task 11 :

Write a function ReverseString and validate its implementation using 5 unittest test cases

```
Task11.py > ...
1 import unittest
2 def reversestring(s):
3     return s[::-1]
4 class Test_reversestring(unittest.TestCase):
5     def test_reversestring(self):
6         self.assertEqual(reversestring("hello"), "olleh")
7     def test_reversestring_empty(self):
8         self.assertEqual(reversestring(""), "")
9     def test_reversestring_single_char(self):
10        self.assertEqual(reversestring("a"), "a")
11    def test_reversestring_palindrome(self):
12        self.assertEqual(reversestring("racecar"), "racecar")
13    def test_reversestring_mixed_case(self):
14        self.assertEqual(reversestring("Hello World"), "dlrow olleH")
15 if __name__ == '__main__':
16     unittest.main()
```

```
PS D:\courses\AIAC\Lab_Ass_8> & C:/Python314/python.exe d:/courses/AIAC/Lab_Ass_8/Task11.py
.....
-----
Ran 5 tests in 0.001s

OK
PS D:\courses\AIAC\Lab_Ass_8> |
```

Task 12 :

Write a function AnagramChecker and validate its implementation using 10 unittest test cases.

```
Task12.py > Test_anagramchecker > test_anagramchecker_with_punctuation
1 import unittest
2 def anagramchecker(s1, s2):
3     return sorted(s1.lower()) == sorted(s2.lower())
4 class Test_anagramchecker(unittest.TestCase):
5     def test_anagramchecker(self):
6         self.assertTrue(anagramchecker("listen", "silent"))
7     def test_anagramchecker_false(self):
8         self.assertFalse(anagramchecker("hello", "world"))
9     def test_anagramchecker_empty(self):
10        self.assertTrue(anagramchecker("", ""))
11    def test_anagramchecker_case_insensitive(self):
12        self.assertTrue(anagramchecker("Listen", "Silent"))
13    def test_anagramchecker_different_lengths(self):
14        self.assertFalse(anagramchecker("hello", "helloo"))
15    def test_anagramchecker_same_letters(self):
16        self.assertTrue(anagramchecker("triangle", "integral"))
17    def test_anagramchecker_non_anagram(self):
18        self.assertFalse(anagramchecker("apple", "pabble"))
19    def test_anagramchecker_with_spaces(self):
20        self.assertTrue(anagramchecker("the eyes", "they see"))
21    def test_anagramchecker_with_punctuation(self):
22        self.assertTrue(anagramchecker("A gentleman", "elegant man"))
23    def test_anagramchecker_with_numbers(self):
24        self.assertFalse(anagramchecker("123", "3214"))
25 if __name__ == '__main__':
26     unittest.main()
```

```
PS D:\courses\AIAC\Lab_Ass_8> & C:/Python314/python.exe d:/courses/AIAC/Lab_Ass_8/Task12.py
.....
-----
Ran 10 tests in 0.002s

OK
PS D:\courses\AIAC\Lab_Ass_8> |
```

Task 13 :

Write a function `ArmstrongChecker` and validate its implementation using 8 unittest test cases.

```
Task13.py > armstrongchecker
1  import unittest
2  def armstrongchecker(n):
3      if n < 0:
4          return False
5      num_str = str(n)
6      num_digits = len(num_str)
7      total = sum(int(digit) ** num_digits for digit in num_str)
8      return total == n
9  class Test_armstrongchecker(unittest.TestCase):
10     def test_armstrong_153(self):
11         self.assertTrue(armstrongchecker(153))
12     def test_non_armstrong_123(self):
13         self.assertFalse(armstrongchecker(123))
14     def test_single_digit(self):
15         self.assertTrue(armstrongchecker(5))
16     def test_zero(self):
17         self.assertTrue(armstrongchecker(0))
18     def test_large_armstrong(self):
19         self.assertTrue(armstrongchecker(9474))
20     def test_large_non_armstrong(self):
21         self.assertFalse(armstrongchecker(9475))
22     def test_negative_number(self):
23         self.assertFalse(armstrongchecker(-153))
24     def test_one_digit_nine(self):
25         self.assertTrue(armstrongchecker(9))
26  if __name__ == '__main__':
27     unittest.main()
28
```

```
PS D:\courses\AIAC\Lab_Ass_8> & C:/Python314/python.exe d:/courses/AIAC/Lab_Ass_8/Task13.py
.....
-----
Ran 8 tests in 0.001s

OK
PS D:\courses\AIAC\Lab_Ass_8>
```